

Урок 6. Корутины. Job

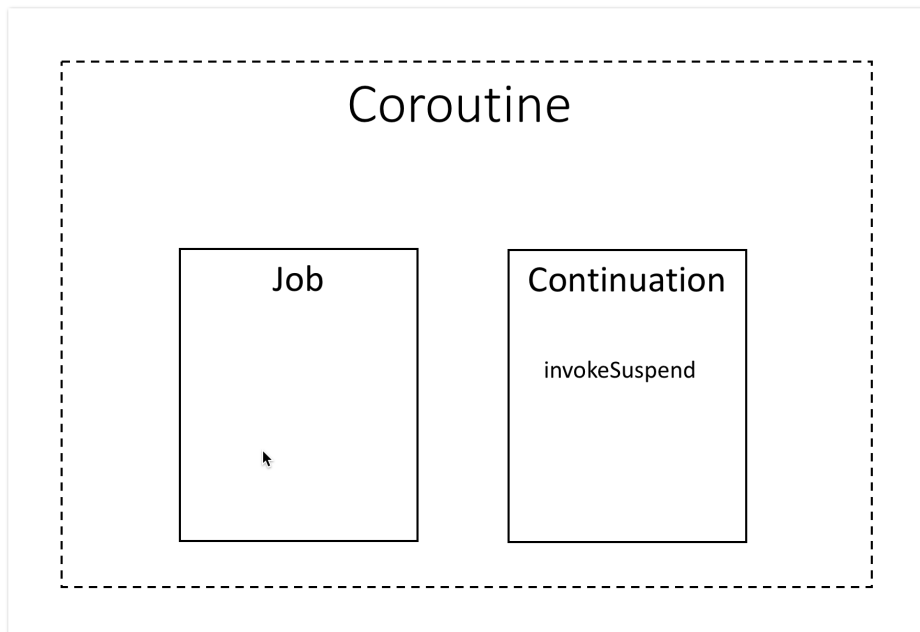
20 декабря 2019

В этом уроке начинаем разбираться, что из себя представляет корутина. Из каких объектов она состоит и что происходит во время ее выполнения.

Подробно разобравшись в том, как работают Continuation и suspend функции, мы можем перейти на уровень выше и рассмотреть, что из себя представляет корутина.

Полная схема корутины сложна и содержит компоненты, которые мы еще не знаем: Job, Context, Dispatcher и CoroutineScope. Поэтому мы начнем с упрощенного варианта схемы, в котором используются только Continuation и Job. А дальше по мере изучения остальных компонентов, будем добавлять их в схему.

Максимально упрощенная схема корутины выглядит так:



Т.е. корутина - это не какой-то конкретный объект. Это набор объектов, которые взаимодействуют друг с другом. Пока что мы рассматриваем только два из них: Continuation и Job.

Про Continuation мы уже многое знаем из прошлых уроков. Он был создан на основе блока кода, который мы передали в корутину. Основная его работа проходит в методе invokeSuspend.

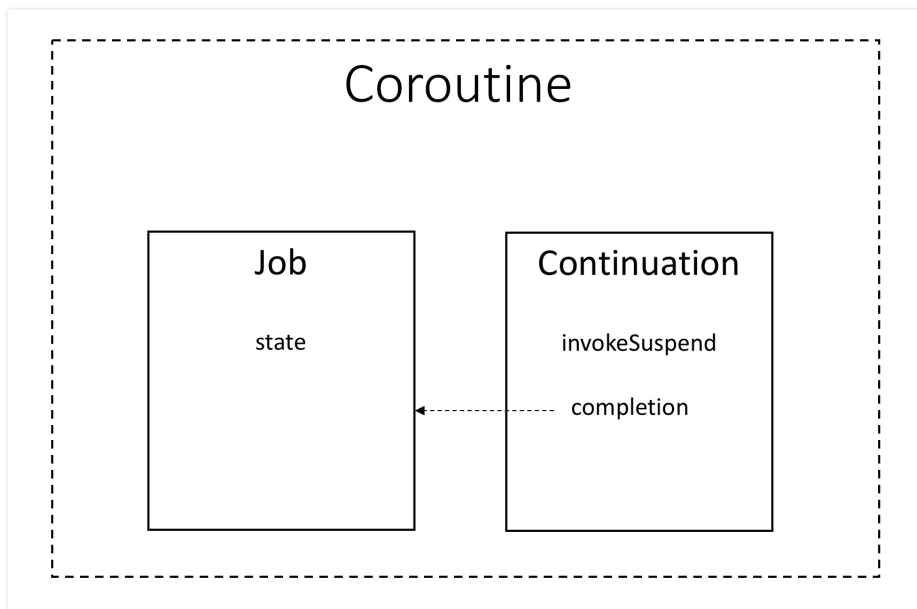
Давайте рассмотрим, что такое Job.

Job

Job является ключевым объектом для корутины. Он выполняет несколько важных функций. Нас пока интересует только одна из них - Job хранит в себе состояние корутины: активна/отменена/завершена. В штатном режиме эти состояния меняются по мере работы корутины. Но мы также иногда можем повлиять на состояние корутины самостоятельно. Например, Job дает нам возможность отменить корутину или стартовать отложенную корутину. Эти операции меняют ее состояние.

Если смотреть в исходники корутин, то там Job - это класс, в котором реализован весь необходимый для его работы базовый функционал. У этого класса есть наследники, в именах которых фигурирует слово Coroutine: StandaloneCoroutine, LazyStandaloneCoroutine, DeferredCoroutine и т.д. Эти наследники Job расширяют поведение базового Job класса и используются в качестве Job в корутине. А какой именно из них будет использован в этом качестве - зависит от билдера корутины и его входных параметров.

Давайте обновим схему корутины с учетом новых знаний о Job.



В Job мы добавляем поле state, в нем хранится текущее состояние корутины.

В Continuation добавляем поле completion, в котором будет ссылка на Job корутины. Когда Continuation закончит выполнение кода в invokeSuspend, он сообщит об этом в Job и тот поменяет состояние корутины.

Lifecycle

Давайте поговорим про жизненный цикл корутины. Он состоит из трех этапов:

- создание
- запуск
- завершение

Рассмотрим простой пример с билдером launch:

```
1 launch {  
2     // coroutine code  
3 }
```

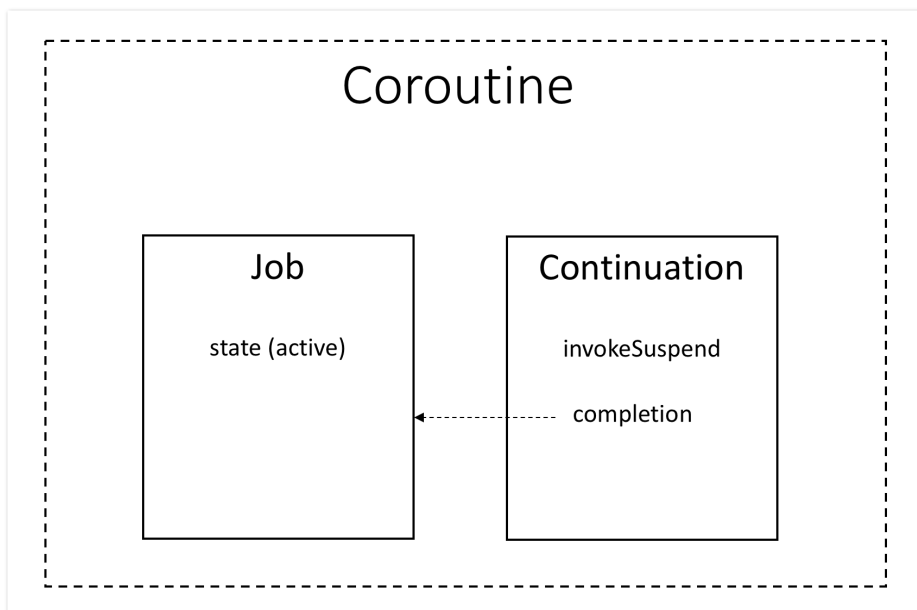
Билдер выполняет первые 2 этапа: создание и запуск.

1. Создание объектов корутины

Билдер в зависимости от себя и своих входных параметров выбирает одну из реализаций Job и создает ее объект. Состояние корутины в нем устанавливается в Активна.

Из блока кода создается Continuation. Его completion содержит ссылку на ранее созданный Job.

На схеме это выглядит так:



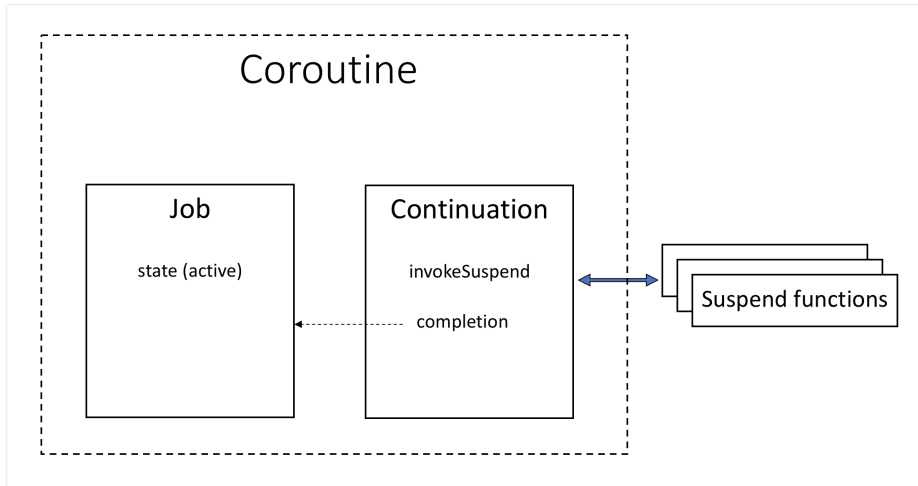
2. Запуск корутины

У Continuation вызывается метод `resume`. Мы подробно обсуждали в прошлых уроках, что `suspend` функции вызывают метод `continuation.resume`, чтобы продолжить работу Continuation. А билдер вызывает этот же самый метод, чтобы работу начать. Метод `resume` в свою очередь запускает `invokeSuspend` и код корутины начинает выполняться.

На этом работа билдера завершена. Он создал и запустил корутину.

Далее корутина живет своей жизнью. Если в Continuation вызываются какие-либо `suspend` функции, то выполнение передается им, а они возвращают его обратно в Continuation по завершении работы.

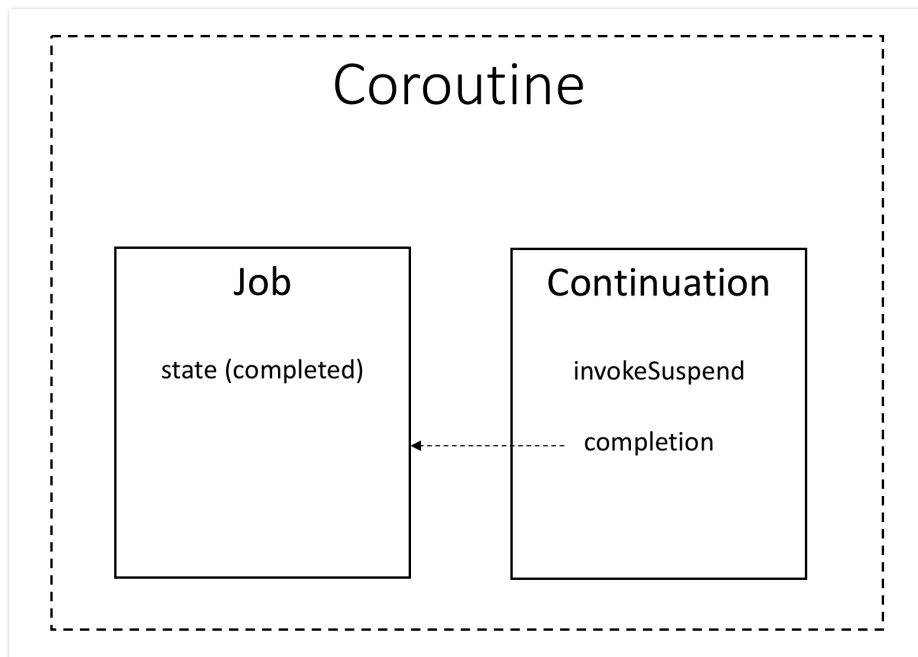
На схеме `suspend` функции можно отобразить так:



Я уже говорил об этом, но повторяюсь. Очень важно в `suspend` функции не потерять вызов `continuation.resume`, иначе корутина просто не продолжит выполняться и никогда не завершится.

3. Завершение

Когда весь код метода `invokeSuspend` выполнен, Continuation сообщает об этом джобу из поля `completion`. Состояние корутины меняется на Завершена.



Если кто-то был подписан на ожидание завершения корутины, то он получит уведомление об этом.

Эта схема и жизненный цикл являются упрощенными. Тут ничего нет про потоки, отмену, родителей, детей, Context, Scope и пр. Все это мы постепенно будем добавлять в последующих уроках.

Как получить Job

Билдер возвращает нам Job

```
1 | val job = launch {
2 |     // coroutine code
3 | }
```



Т.е. этот тот самый Job, который был создан билдером и используется в корутине.

Мы можем использовать его, чтобы отменить корутину, запустить отложенную корутину, дождаться завершения корутины. Все эти операции мы еще рассмотрим подробно в последующих уроках.